Mansi Davkhar
D20C / 13

Blockchain Experiment No: 8

Aim

To configure a cryptocurrency wallet (MetaMask) and integrate with a Decentralized Application (Dapp) designed for optimal and secure Banking Transactions on the Blockchain.

Theory

MetaMask

MetaMask is a browser-extension (and mobile) cryptocurrency wallet that stores a user's private keys and lets the browser talk to the Ethereum blockchain. It signs every transaction on the user's behalf after the user approves it, so websites and DApps never see the private key. It can connect to the Ethereum mainnet, test networks such as Sepolia, and custom RPC networks.

Test Network (Testnet)

A testnet is a copy of a blockchain that works like the main network but uses coins that have no real-world value. Developers use it to deploy and test smart contracts without spending real money. Test coins are obtained free from a faucet. Sepolia is a widely used Ethereum proof-of-stake testnet, and RSK Testnet is the equivalent for the RSK network.

Remix IDE

Remix is a browser-based IDE for writing, compiling, deploying and debugging Solidity smart contracts. Choosing the environment "Injected Provider - MetaMask" in the Deploy & Run Transactions tab makes Remix send all transactions through MetaMask, so the account and network selected in MetaMask are used.

DApp and ethers.js

A decentralized application (DApp) has a web front end that reads from and writes to a smart contract on a blockchain instead of a central server. In this experiment the front end is a single HTML page that uses the ethers.js library. The page asks MetaMask for the user's account (eth_requestAccounts), creates a Contract object from the contract address and its ABI, and then calls the contract's functions. Read-only (view) functions are free; state-changing functions open a MetaMask pop-up where the user confirms and pays gas.

Smart contract used: CertRegistry

CertRegistry is an on-chain registry for academic certificates. The certificate file is encrypted and stored off-chain on IPFS, and only two small values are stored on the blockchain: the IPFS content identifier (CID) and the keccak256 hash of the file. Any change to the file later produces a different hash, so tampering is detected. The main functions are:

- **issueCertificate(certId, cid, dataHash):** records a certificate; callable only by an authorized issuer.
- **revokeCertificate(certId, reason):** marks a certificate as revoked; callable only by its original issuer.
- **verifyCertificate(certId, dataHash):** a free view call that returns NotFound, Valid, Tampered or Revoked.
- **addIssuer / removeIssuer:** owner-only functions to manage authorized issuers.

Tasks Performed

Part A: Set up MetaMask

A new wallet was created with a password, and the Secret Recovery Phrase was written down and stored safely. (An existing wallet can be imported with its recovery phrase instead.) The recovery phrase and private key must never be shared or shown in the screenshots.

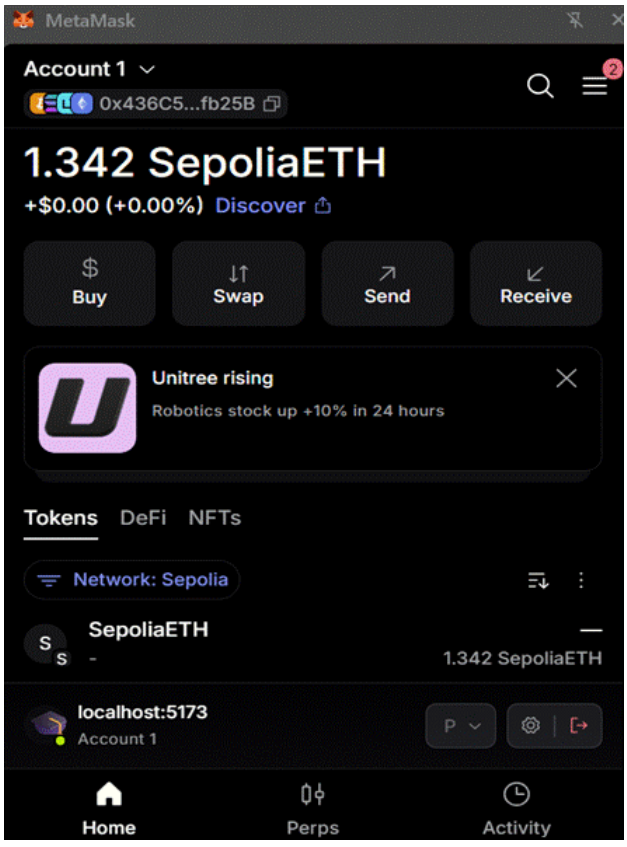


Fig. 1: MetaMask wallet created, showing the account (hide the recovery phrase)

Part B: Create, compile and deploy the smart contract

In the Solidity Compiler tab, compiler version 0.8.24 or higher was selected and the contract CertRegistry.sol was compiled. A green tick on the compiler icon confirmed a successful compilation.

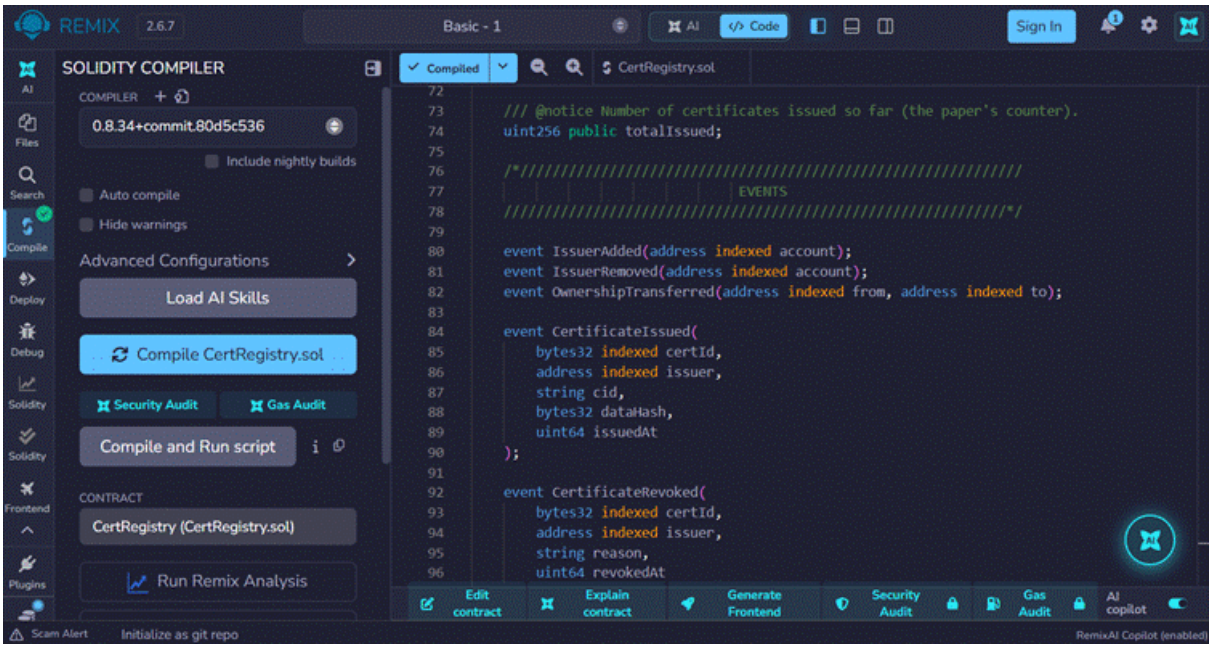


Fig. 2: Successful compilation of CertRegistry.sol

In the Deploy & Run tab, the contract CertRegistry was selected and Deploy was clicked (the constructor takes no arguments). MetaMask opened a confirmation pop-up showing the estimated gas fee; the transaction was confirmed. After mining, the deployed contract appeared under "Deployed Contracts" and its address was copied.

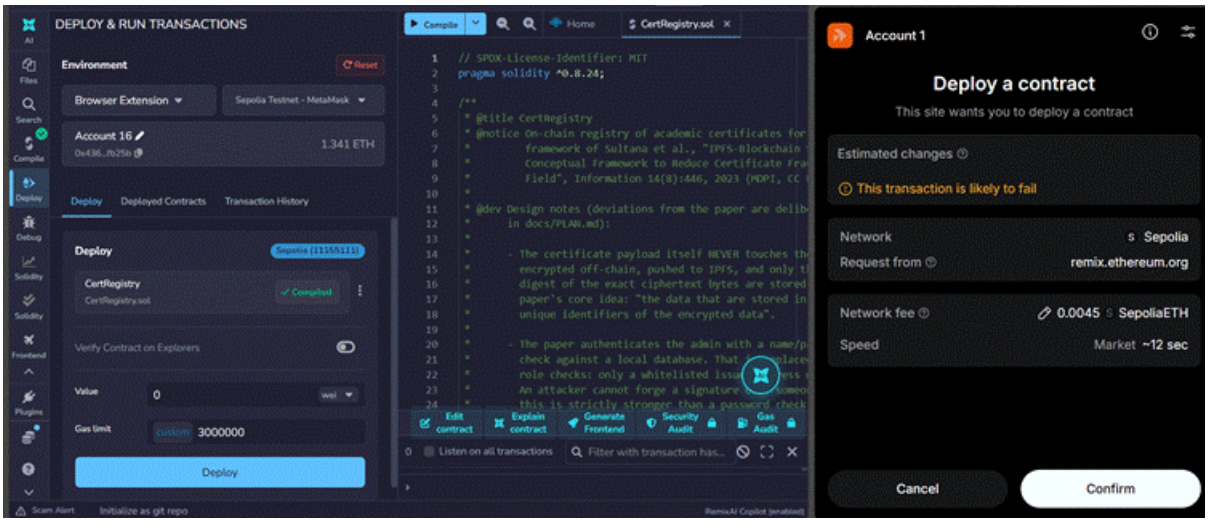


Fig. 3: MetaMask confirmation pop-up for contract deployment

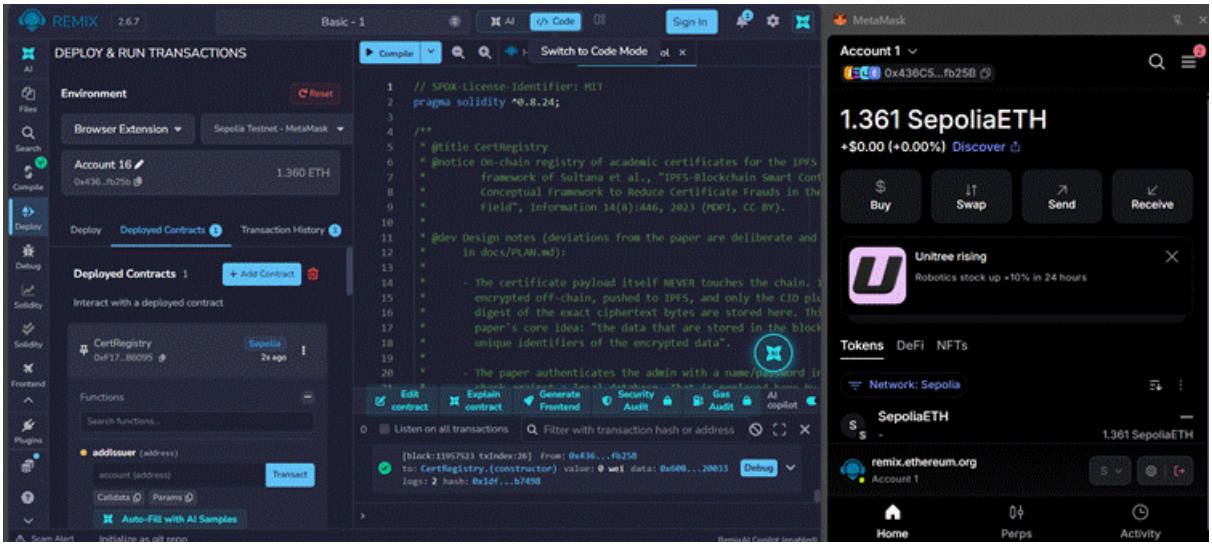


Fig. 4: Deployed CertRegistry shown in Remix with contract address

Part C: Check the transaction on the block explorer

The transaction hash of the deployment was copied from the Remix terminal (or from MetaMask Activity) and searched on the Sepolia block explorer (sepolia.etherscan.io). The explorer shows the status, block number, sender, the created contract address, gas used and transaction fee. (If RSK Testnet was used, the RSK Explorer is used in the same way.)

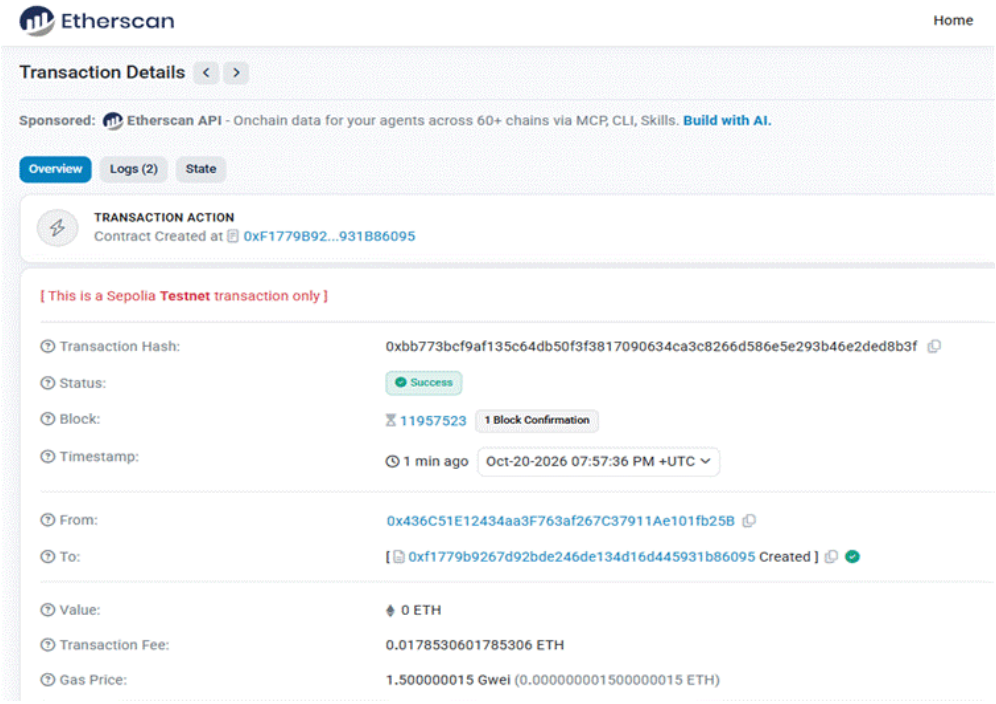


Fig. 5: Deployment transaction details on the block explorer

Part D: Interact with the smart contract in Remix

Using the deployed contract panel in Remix, the following calls were made. Each state-changing call opened a MetaMask confirmation; each view call returned instantly without gas.

1. computeCertId(studentId, course, issueDate) was called to obtain the certificate ID (free call).
2. issueCertificate(certId, cid, dataHash) was sent with a sample IPFS CID and a 32-byte hash, and confirmed in MetaMask.
3. verifyCertificate(certId, dataHash) was called with the correct hash and returned status 1 (Valid).
4. verifyCertificate was called again with a different hash and returned status 2 (Tampered).
5. revokeCertificate(certId, reason) was sent and confirmed; verifyCertificate then returned status 3 (Revoked).

Part E: Integrate the MetaMask wallet with a DApp

The same contract was then used from a web page instead of Remix. The page (index.html, listed in the Program section) uses ethers.js to talk to MetaMask and the deployed CertRegistry. The steps were:

1. Save the DApp code as index.html in a folder. Open a terminal in that folder and start a local web server, for example: npm run dev. Open http://localhost:5173 in the browser that has MetaMask installed.
2. Paste the deployed CertRegistry address into the page and click "Connect MetaMask". MetaMask asks for permission to connect; approve it. The page displays the account address, network, ETH balance and whether the account is an issuer.
3. Issue a certificate: enter Student ID, Course, Issue date, an IPFS CID and choose a certificate file. The page computes the certificate ID and the keccak256 hash of the file in the browser, calls issueCertificate, and MetaMask asks the user to confirm the transaction.
4. Verify a certificate: enter the same details and choose the same file. The page calls verifyCertificate (no gas, no pop-up) and shows Valid.
5. Revoke the certificate from the page, confirm in MetaMask, then verify again. The page shows Revoked.

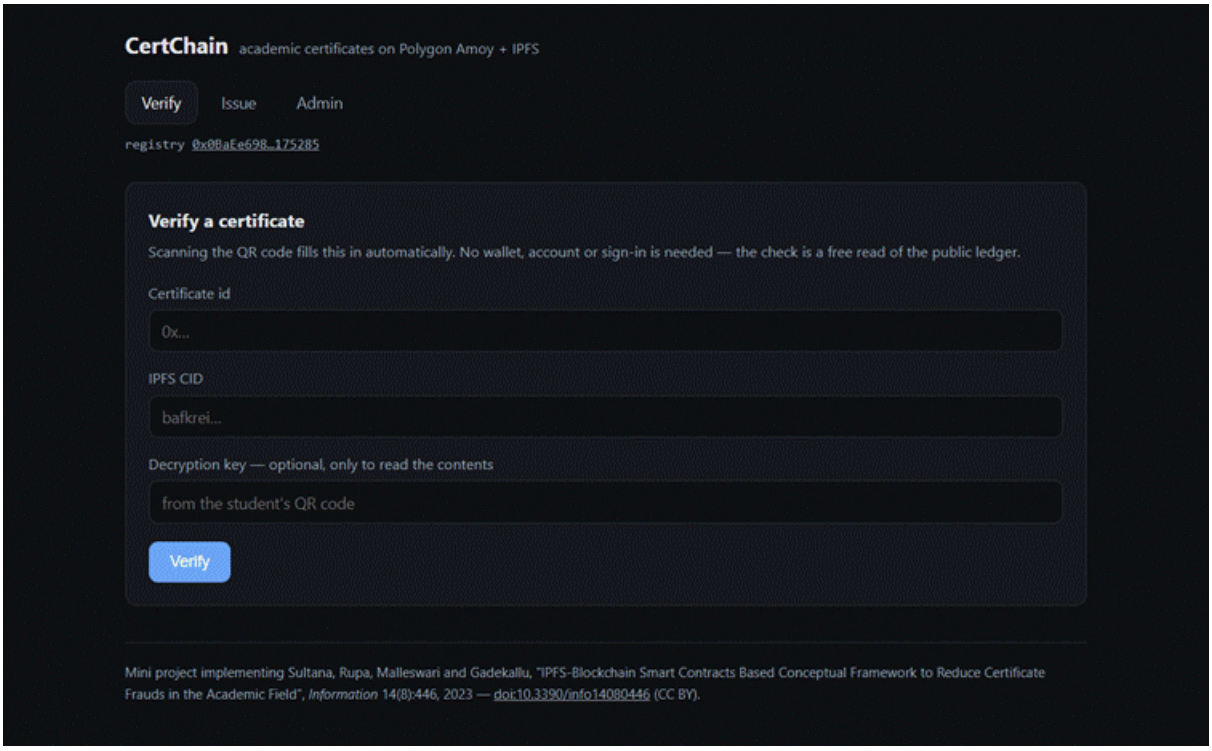


Fig. 6: DApp page loaded at localhost with contract address entered

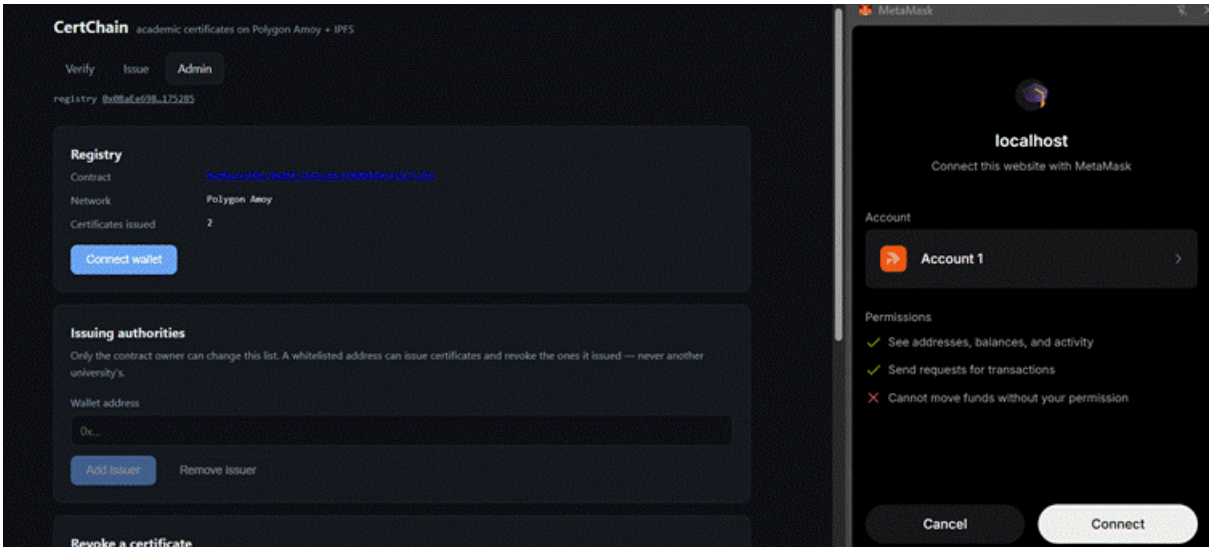


Fig. 7: MetaMask connection request from the DApp

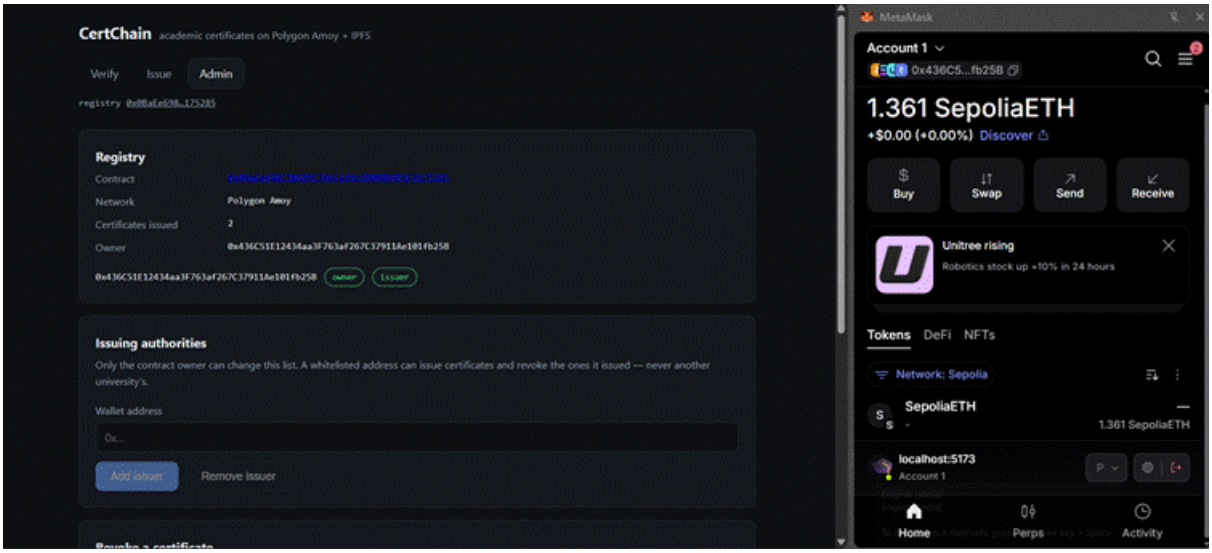


Fig. 8: DApp showing connected account, network and balance

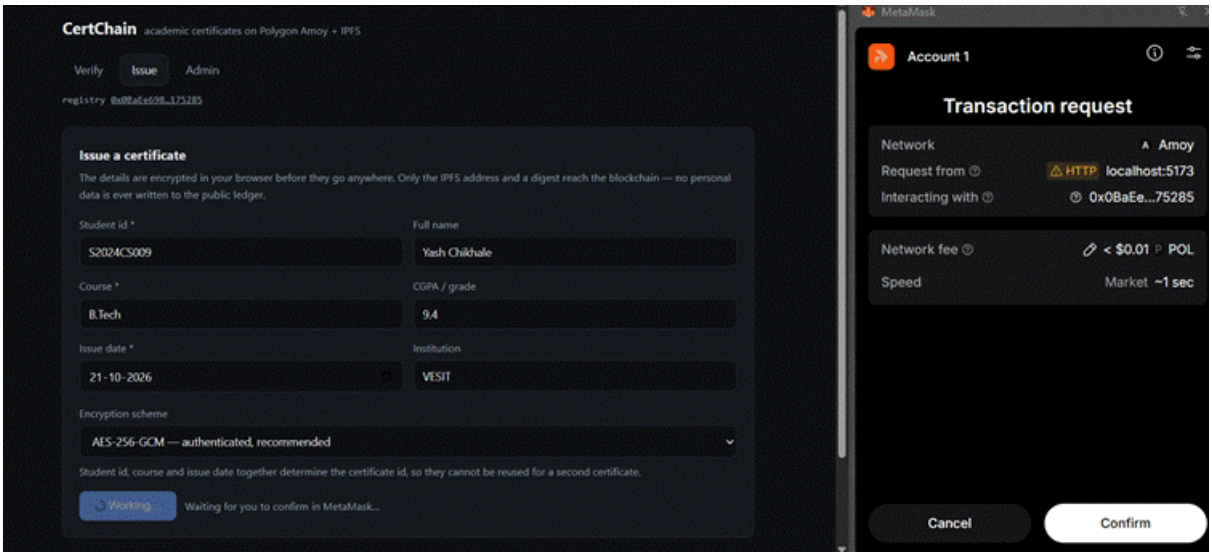


Fig. 9: Issue certificate: MetaMask transaction confirmation from the DApp

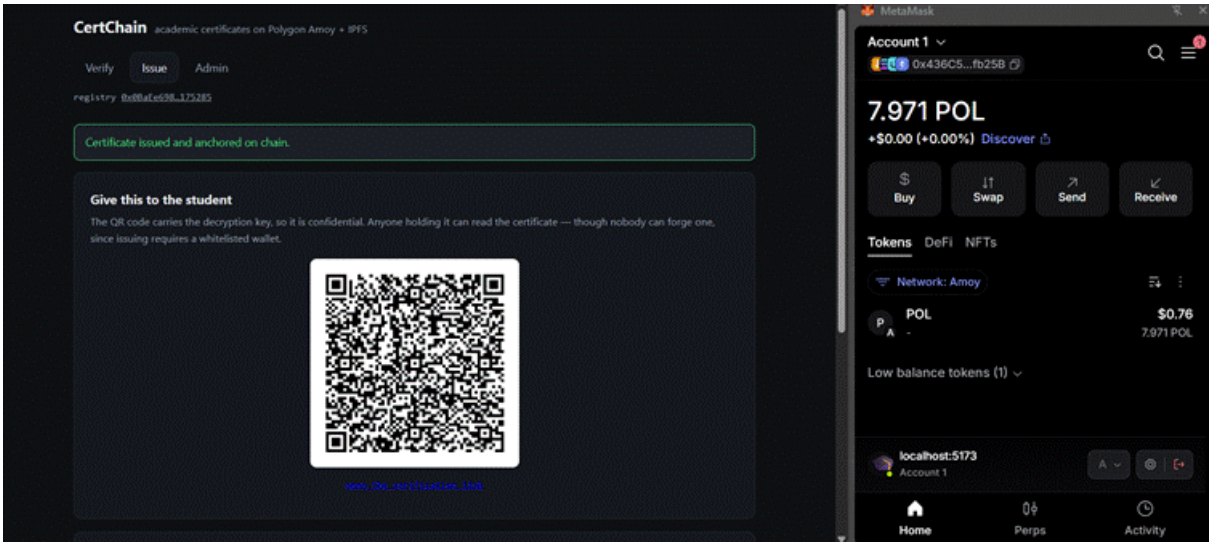


Fig. 10: DApp output after issuing the certificate (tx hash, block, cert ID)

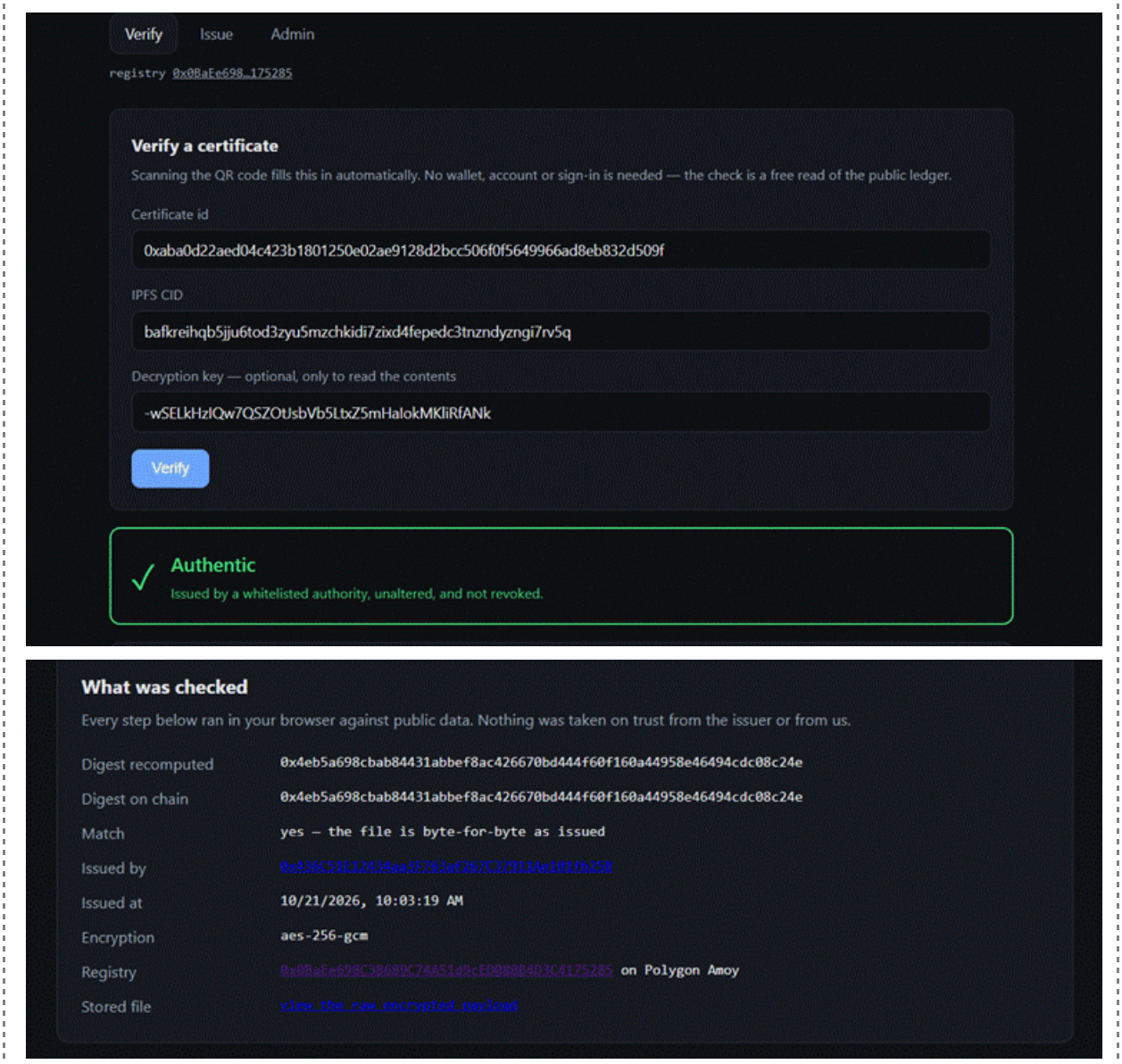


Fig. 11: DApp verification results: Valid and Tampered

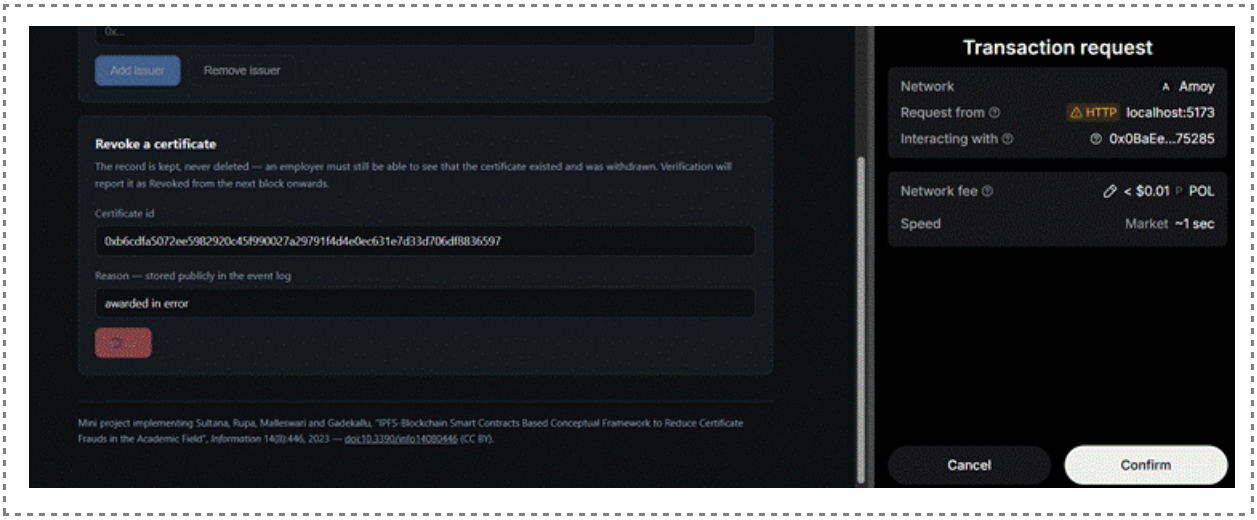


Fig. 12: DApp output after revoking, and verification showing Revoke

Inference

- The contract deployed successfully on the local Ganache blockchain, and each state-changing call (issue, revoke, add issuer) appeared as a mined transaction with gas used and a reduced account balance.
- Read-only calls such as verifyCertificate() cost no gas and created no transactions.
- Only the hash and CID are stored on-chain, so the certificate data stays private.
- Verification returned *Valid* for the original hash and *Tampered* for a modified one. After revocation it returned *Revoked*.
- Access control worked: non-issuer accounts could not issue or revoke certificates.

Conclusion

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with Metamask and Remix, the CertRegistry contract was deployed, tested and verified without spending real Ether. The experiment showed how blockchain can reduce certificate fraud by providing tamper-proof, publicly verifiable records.